Python Tutorial for Beginners

Welcome to the Python tutorial for beginners! This guide is designed to help you learn Python from scratch. We'll cover the basics, provide coding examples, and include quiz questions to test your understanding. By the end of this tutorial, you'll be able to write

simple Python programs and have a strong foundation upon which to build. Let's get started!

Introduction to Python **4**
What is Python? 4
Why Learn Python? 4
Setting Up Python **4**
Download and Install Python 4
Verify Installation 4
Installing a Code Editor 5
Your First Python Program **5**
Hello, World! 5
Quiz Question 5
Basic Syntax **6**
Indentation 6
Comments 6
Variables 7
Quiz Question 7
Variables and Data Types **7**
Variable Assignment 7
Data Types 7
Type Casting 8
Quiz Question 8
Operators **8**
Arithmetic Operators 8
Assignment Operators 9
Comparison Operators 9
Logical Operators 9
Quiz Question 10
Control Flow **10**
If Statements 10
Elif Statements 10
Nested If Statements 11
Loops 11
While Loop 11
For Loop 11
Break and Continue 11

Quiz Question 12
Functions **12**
Defining a Function 12
Calling a Function 12
Default Parameters 12
Keyword Arguments 13
Variable-Length Arguments 13
Quiz Question 13
Lists and Tuples **14**
Lists 14
Tuples 14
Quiz Question 15
Dictionaries **15**
Creating a Dictionary 15
Accessing Values 15
Modifying Dictionaries 15
Looping Through a Dictionary 15
Quiz Question 16
Modules and Packages **16**
Importing Modules 16
Importing Specific Functions 16
Creating a Module 16
Quiz Question 17
File Handling **17**
Opening a File 17
Reading a File 18
Writing to a File 18
Using with Statement 18
Quiz Question 18
Exception Handling **18**
Basic Try-Except 19
Catching Multiple Exceptions 19
Finally Block 19
Raising Exceptions 19
Quiz Question 20
Conclusion **20**
Final Quiz **20**

Introduction to Python

What is Python?

Python is a high-level, interpreted programming language known for its readability and simplicity. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming.

Why Learn Python?

- **Easy to Learn**: Simple syntax that's beginner-friendly.
- **Versatile**: Used in web development, data science, machine learning, automation, and more.
- **Large Community**: Extensive libraries and frameworks available.
- **High Demand**: Widely used in the industry with many job opportunities.

Setting Up Python

Download and Install Python

1. Visit the official Python website.
2. Download the latest version of Python (e.g., Python 3.x).
3. Run the installer and follow the prompts.
    - **Windows Users**: Ensure that you check the "Add Python to PATH" option during installation.

Verify Installation

Open your terminal or command prompt and type:

```
python --version
```

This should display the installed Python version.

Installing a Code Editor

Choose a code editor or IDE such as:

- **Visual Studio Code**
- **PyCharm**
- **Sublime Text**
- **Atom**

Your First Python Program

Let's write a simple Python program.

Hello, World!

1. Open your code editor.
2. Create a new file and save it as `hello.py`.

hello.py

```
print("Hello, World!")
```

3. Run the program:
    - In the terminal, navigate to the directory containing `hello.py`.

Type:
```
python hello.py
```

    -

Output:
```
Hello, World!
```

4.

Quiz Question

Q1: Which function is used to display output in Python?

A. `echo()`
B. `print()`
C. `display()`
D. `show()`

Answer: B. `print()`

Basic Syntax

Python syntax is designed to be readable and clean.

Indentation

- Python uses indentation (whitespace) to define code blocks.
- Typically, four spaces are used for indentation.

Example:

```
if 5 > 2:
    print("Five is greater than two.")
```

Comments

Single-line comment: Use #

```
# This is a comment
```

-

Multi-line comments: Use triple quotes `'''` or `"""`

```
"""
This is a
multi-line comment
"""
```

-

Variables

- Variables do not need explicit declaration.
- Variable names are case-sensitive.

Example:

```
name = "Alice"
age = 25
```

Quiz Question

Q2: How do you write a single-line comment in Python?

A. `// This is a comment`
B. `/* This is a comment */`
C. `# This is a comment`
D. `<-- This is a comment -->`

Answer: C. `# This is a comment`

Variables and Data Types

Variable Assignment

```
x = 10
y = "Hello"
```

Data Types

- **Numeric Types**: `int`, `float`, `complex`
- **Text Type**: `str`

- **Boolean Type**: `bool`
- **Sequence Types**: `list`, `tuple`, `range`
- **Mapping Type**: `dict`
- **Set Types**: `set`, `frozenset`
- **None Type**: `NoneType`

Type Casting

Convert between data types using functions like `int()`, `float()`, `str()`.

Example:

```
num_str = "50"
num_int = int(num_str)
```

Quiz Question

Q3: What data type is the following value: `True`?

A. `str`
B. `int`
C. `bool`
D. `NoneType`

Answer: C. `bool`

Operators

Arithmetic Operators

- Addition: `+`
- Subtraction: `-`
- Multiplication: `*`
- Division: `/`

- Modulus: %
- Exponentiation: **
- Floor Division: //

Example:

```
a = 10
b = 3
print(a + b)    # 13
print(a % b)    # 1
```

Assignment Operators

```
x = 5
x += 3  # Equivalent to x = x + 3
```

Comparison Operators

- Equal: ==
- Not equal: !=
- Greater than: >
- Less than: <
- Greater than or equal to: >=
- Less than or equal to: <=

Logical Operators

- `and`
- `or`
- `not`

Example:

```
x = 5
print(x > 3 and x < 10)  # True
```

Quiz Question

Q4: What is the output of `print(2 ** 3)`?

A. 5
B. 6
C. 8
D. 9

Answer: C. 8

Control Flow

Control flow statements allow you to control the execution of code.

If Statements

```
age = 18
if age >= 18:
    print("You are an adult.")
else:
    print("You are a minor.")
```

Elif Statements

```
score = 85
if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
else:
    print("Grade: C")
```

Nested If Statements

```
num = 10
if num > 0:
    print("Positive")
    if num % 2 == 0:
        print("Even")
```

Loops

While Loop

```
count = 0
while count < 5:
    print(count)
    count += 1
```

For Loop

```
for i in range(5):
    print(i)
```

Break and Continue

- **break**: Exit the loop
- **continue**: Skip to the next iteration

Example:

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

Quiz Question

Q5: What is the output of the following code?

```
for i in range(3):
    print(i)
```

A. `1 2 3`
B. `0 1 2`
C. `0 1 2 3`
D. `1 2`

Answer: B. `0 1 2`

Functions

Functions are reusable blocks of code.

Defining a Function

```
def greet(name):
    return f"Hello, {name}!"
```

Calling a Function

```
message = greet("Alice")
print(message)  # Outputs: Hello, Alice!
```

Default Parameters

```
def greet(name="Guest"):
    return f"Hello, {name}!"
```

Keyword Arguments

```
def add(a, b):
    return a + b

result = add(b=3, a=5)
```

Variable-Length Arguments

- ***args**: Non-keyword variable arguments
- ****kwargs**: Keyword variable arguments

Example:

```
def multiply(*args):
    result = 1
    for num in args:
        result *= num
    return result

print(multiply(2, 3, 4))  # Outputs: 24
```

Quiz Question

Q6: How do you define a function in Python?

A. `function myFunc():`
B. `def myFunc():`
C. `function:myFunc()`
D. `def myFunc:`

Answer: B. `def myFunc():`

Lists and Tuples

Lists

- Ordered, mutable collections.

Creating a List:

```
fruits = ["apple", "banana", "cherry"]
```

Accessing Elements:

```
print(fruits[0])  # Outputs: apple
```

Modifying Lists:

```
fruits.append("date")
fruits.remove("banana")
```

List Comprehensions:

```
squares = [x**2 for x in range(5)]
```

Tuples

- Ordered, immutable collections.

Creating a Tuple:

```
coordinates = (10, 20)
```

Accessing Elements:

```
print(coordinates[1])  # Outputs: 20
```

Quiz Question

Q7: Which of the following is a tuple?

A. `["apple", "banana", "cherry"]`
B. `("apple", "banana", "cherry")`
C. `{"apple", "banana", "cherry"}`
D. `{"name": "apple", "color": "red"}`

Answer: B. `("apple", "banana", "cherry")`

Dictionaries

Dictionaries are unordered collections of key-value pairs.

Creating a Dictionary

```
person = {
    "name": "Alice",
    "age": 30,
    "city": "New York"
}
```

Accessing Values

```
print(person["name"])  # Outputs: Alice
```

Modifying Dictionaries

```
person["age"] = 31
person["email"] = "alice@example.com"
```

Looping Through a Dictionary

```
for key, value in person.items():
    print(f"{key}: {value}")
```

Quiz Question

Q8: How do you access the value associated with the key `"age"` in the dictionary `person`?

A. `person["age"]`
B. `person(age)`
C. `person.get("age")`
D. Both A and C

Answer: D. Both A and C

Modules and Packages

Modules are files containing Python code; packages are collections of modules.

Importing Modules

```
import math

print(math.sqrt(16))  # Outputs: 4.0
```

Importing Specific Functions

```
from math import pi

print(pi)  # Outputs: 3.141592653589793
```

Creating a Module

1. Create a file named `my_module.py`.

my_module.py

```
def greet(name):
    return f"Hello, {name}!"
```

2. Use the module in another file.

main.py

```
import my_module

print(my_module.greet("Alice"))
```

Quiz Question

Q9: How do you import the `random` module in Python?

A. `include random`
B. `import random`
C. `using random`
D. `require random`

Answer: B. `import random`

File Handling

Read from and write to files.

Opening a File

```
file = open("example.txt", "r")  # Modes: r, w, a, r+, etc.
```

Reading a File

```
content = file.read()
print(content)
file.close()
```

Writing to a File

```
file = open("example.txt", "w")
file.write("Hello, File!")
file.close()
```

Using `with` Statement

Automatically handles file closing.

```
with open("example.txt", "r") as file:
    content = file.read()
```

Quiz Question

Q10: Which method is used to read the entire contents of a file?

A. `file.readlines()`
B. `file.read()`
C. `file.readline()`
D. `file.readfile()`

Answer: B. `file.read()`

Exception Handling

Handle errors gracefully using try-except blocks.

Basic Try-Except

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero.")
```

Catching Multiple Exceptions

```
try:
    # Code that may raise exceptions
    pass
except (TypeError, ValueError) as e:
    print(f"An error occurred: {e}")
```

Finally Block

Executed regardless of whether an exception occurs.

```
try:
    # Code
    pass
except Exception:
    # Handle exception
    pass
finally:
    # Always executed
    print("Cleaning up.")
```

Raising Exceptions

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative.")
```

```
    else:
        print(f"Age is set to {age}.")

set_age(-5)
```

Quiz Question

Q11: What keyword is used to handle exceptions in Python?

A. `catch`
B. `except`
C. `handle`
D. `error`

Answer: B. `except`

Conclusion

Congratulations! You've covered Python programming basics, including variables, data types, operators, control flow, functions, data structures, modules, file handling, and exception handling. With this foundation, you're well on your way to becoming proficient in Python. Keep practicing and exploring more advanced topics to enhance your skills.

Final Quiz

Q12: Which of the following is NOT a valid Python data type?

A. `list`
B. `tuple`
C. `array`
D. `dictionary`

Answer: C. `array`

Q13: How do you start a for loop in Python?

A. `for (i = 0; i < 5; i++):`
B. `for i in range(5):`
C. `foreach i in 5:`
D. `loop i from 0 to 5:`

Answer: B. `for i in range(5):`

Q14: What is the correct way to define a class in Python?

A. `class MyClass { }`
B. `def class MyClass:`
C. `class MyClass:`
D. `class MyClass():`

Answer: Both C and D are correct (In Python 3, parentheses are optional if not inheriting from a superclass)

Q15: How do you check the type of a variable x?

A. `print(type(x))`
B. `print(typeof x)`
C. `print(class(x))`
D. `print(var(x))`

Answer: A. `print(type(x))`